

Project Pixie: Personalized Cultural & Linguistic Intelligence

An Intelligent Name Analysis Agent

<https://github.com/auj012/AIAgentLearning/tree/main/ProjectPixie>

Ushasree Jakilinki

SW Engineer → Aspiring Data Scientist + Agentic AI Developer
UofM Master's in Applied Data Science (2025)

About Me



25+ Years

SW Engineering
Automotive & Mobile



B Tech CSE

Computer Science
& Engineering



AI/DS Pivot

Aspiring
Data Scientist



UofM MADS

Master's Applied
Data Science 2025

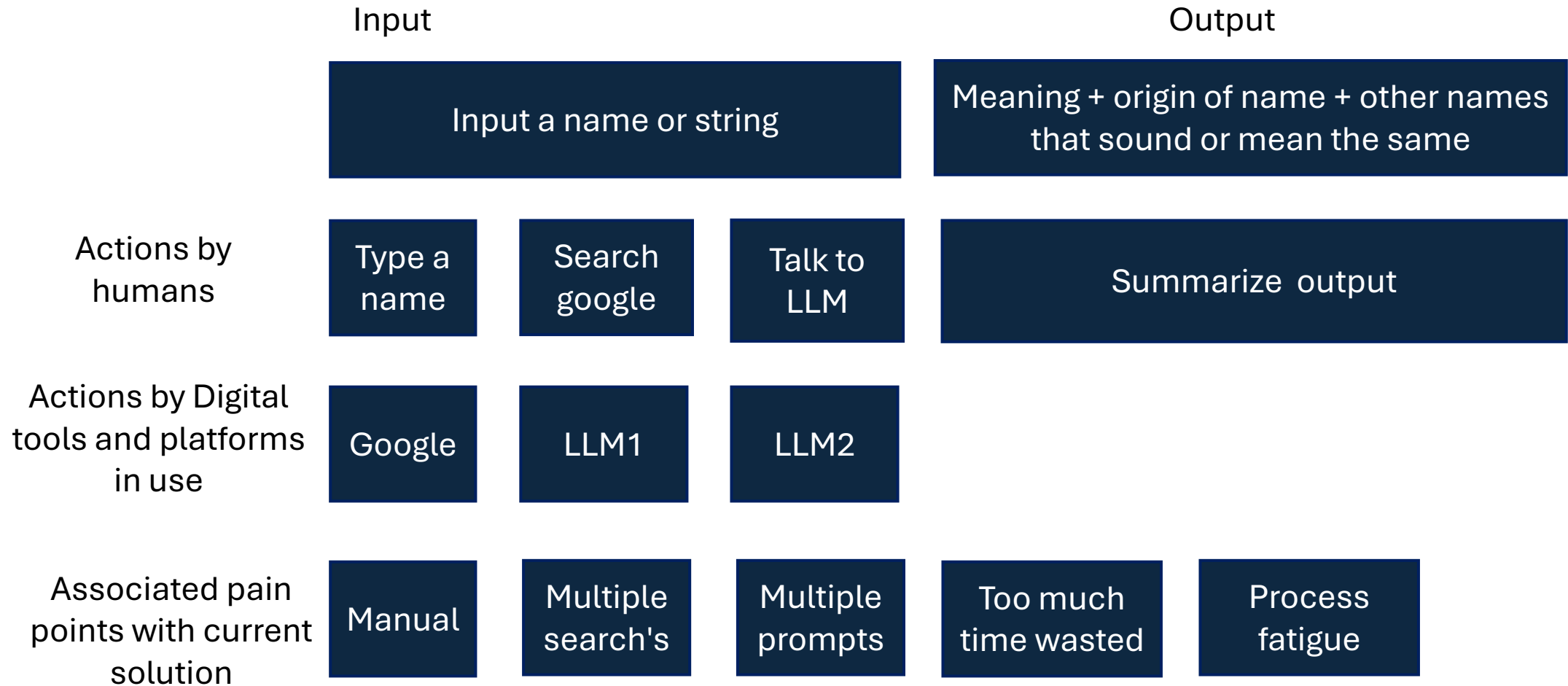
Why Project Pixie?

Applying enterprise-grade software engineering principles to build a tightly scoped, reliable, local-first AI agent utility.

Problem statement and System Objective

- Goal: Build a local AI agent using Ollama
- Input: A single name string
- Output: A highly accurate, structured analytics payload containing:
 - Linguistic Origin Matrix -The geographic and historical language roots.
 - Literal & Semantic Translation-The exact meaning and historical definitions.
 - Cultural Metadata -Historical usage trends, variations, or significant notable contexts.

The Current Process & Pain Points



AI card

Generative AI –Local LLM-
Powered by Ollama to process data
100% offline with zero cloud
subscription costs.

Risk mitigation card

Privacy

Zero data leakage. User target
strings stay entirely local and never
leave the host machine.

Dignity

Cultural guardrails ensure diverse and
ethnic names are analyzed with
historical accuracy, avoiding generic
or biased AI definitions.

Functional and user requirements

As a...	I want to ...	So that...	Acceptance Criteria
User	Type my name or string into the system using keyboard	I can get more information about the name +string	Given a name I get the origin and meaning of the name. In addition I get similar sounding names and similar meaning names as well.
User	Make sure that the system handles profanity well	Anyone can use the system safely	Given a profanity in any language (typed in English) give back some polite feedback.
User	I want the system to respond within 1 sec	A user does not get bored and not move to another tool	Display a time counter to show feedback time.
User	I want the UI/FE to be clear simple and minimalistic	Learning curve is non-existent	Clean, single-input text box layout with zero clutter
Developer	I want to see metrics every week	So I can monitor the tool/system	A report where developer can check report

Guardrails

Rule	System Action
Profanity Check	Drop toxic or offensive words immediately and return polite feedback.
Language Check	Enforce standard A-Z alphabetic text strings (ignoring non-English character sets for MVP).
At Least Typed	Reject empty strings, blank spaces, or null submissions to prevent system errors.

Evals

Evaluation Tier	Testing Method	Success Metric (Acceptance Criteria)
Unit Testing	Isolated programmatic testing of prompt templates, extraction regex, and mock LLM API responses.	Deterministic Pass: Zero dependency on Ollama; ensures code logic and string formatting are mathematically flawless.
Model-Driven-Evaluation(Agent-on-Agent)	Deploy a dedicated Tester Agent to bombard Project Pixie with hundreds of synthetic name inputs.	Consistency Check: Pixie accurately extracts origin and meaning without hallucinating or breaking format over 100+ runs.
Behavioral Evaluation	Adversarial Red-Teaming using a curated test dataset of restricted words and symbols.	100% Block Rate: System properly catches toxic inputs and gracefully drops non-A-Z characters.
Performance (System Test)	Measure latency from the moment the user hits Enter until the local LLM responds.	Speed Benchmark: System successfully returns responses in under 1 second .

System Execution Examples

TEST INPUT STRING	EXECUTION PATH	SYSTEM OUTPUT PAYLOAD RESPONSE
"Priya"	Clean pass → Ollama inference → JSON parse	{"origin": "Sanskrit", "meaning": "Beloved", "variants": ["Preeya"]}
"Kenji"	Clean pass → Ollama inference → JSON parse	{"origin": "Japanese", "meaning": "Intelligent Second Son", "variants": ["Ken"]}
"DROP TABLE Names;"	Intercepted by Guardrail → Bypass LLM	Refusal: <i>"Invalid input. Please enter a valid alphabetic name structure."</i>

Estimates and Schedule

Milestone	Name	Estimate in hours	Responsible person
1	Discovery, Ideation & Requirements	40	Ushasree Jakilinki
2	PRD, Architecture & Core Implementation	30	Ushasree Jakilinki
3	Unit Testing & Model Evaluations (Evals)	30	Ushasree Jakilinki
4	Deployment & Final Validation	30	Ushasree Jakilinki

Metrics +KPIs

Metric	What It Measures	Why It Matters for Project Pixie	Target Goal
Token Generation Latency	Time elapsed from user input to the final generated text token.	Ensures local execution (Ollama) doesn't hang or freeze on standard laptops.	< 10 seconds end-to-end response
Schema/Format Compliance	% of outputs that successfully parse into valid structured data (JSON/Markdown).	If the local LLM outputs raw messy text, your UI code breaks.	100% compliance across unit test suites
Guardrail Rejection Rate	% of adversarial or profane inputs successfully intercepted by your logic layer.	Proves that your regex and safety barriers are successfully protecting the core LLM.	100% intercept on restricted test sets
Hallucination Rate	% of generated outputs containing fabricated or nonsensical origins/meanings.	Measured by your Tester Agent to guarantee high data integrity.	< 5% across automated test batches

Project Risks & Mitigation Strategies

Identified Risk	Impacted KPI / Milestone	PM Mitigation Strategy
1. Local Compute Latency Spike: The local model takes longer than 1.0 second to generate tokens on standard consumer laptops.	Performance Benchmark (< 1.0s response target)	Implement aggressive 4-bit model quantization or drop parameter size (e.g., from a 7B model to a 1.5B/3B model) to preserve processing speed.
2. Schema & Formatting Drift: The local model occasionally outputs raw text or bad markdown, breaking the frontend layout.	Schema/Format Compliance (100% JSON goal)	Use regex extraction fallback blocks in your Python code to isolate the JSON, or utilize structured generation settings in Ollama to force strict schema compliance.
3. Schedule Slippage (Single Resource): Unexpected technical blockers consume hours, threatening the 130-hour timeline.	Schedule & Final Validation	Establish strict feature freezes. If a blocker occurs, drop future extensions (like Multi-Language Support or Voice) to protect the core MVP delivery deadline.

Cost Analysis

- Zero Token Costs: By design, leveraging local open-source models completely eliminates dependencies on third-party cloud APIs (like OpenAI or Anthropic). You can run thousands of test queries for free.
- Hardware Utilization: The only resource utilized is your host machine's local compute power (CPU/GPU and RAM).

Future Extensions

- Multi-Language Support (Localization): Expand the prompt architecture to translate name origins and meanings into multiple target languages based on user preference.
- Multimodal Voice Integration: Implement Speech-to-Text (STT) for voice-activated inputs and Text-to-Speech (TTS) for audio-based output generation.
- Historical Database Caching: Store previously searched names in a lightweight local database (like SQLite) to eliminate duplicate LLM inference calls and make response times instant for popular names.

Questions & Discussion

- *Open for technical feedback, architectural critiques, or feature suggestions.*
- *Ushasree Jakilinki: auj012@gmail.com; ushajaki@umich.edu*